MTG CARD VAULT API

PREPARED FOR **Chris**

Stack —	Due —	Time —	Endpoints —
C# · ASP.NET Core · EF Core · SQLite	First class session after break	10–14 hours across the week	6 (5 CRUD + 1)

THE BRIEF

A Magic: The Gathering player has a few thousand cards in long boxes and no idea what any of it is worth. They want an API that tracks every card they own: which set it came from, how many copies, what it costs to cast, and what it is worth, with a quick way to see the value of everything they hold from a single set. Every row is a **Card** in the collection.

/01

WHAT YOU ARE BUILDING

Everything in this handout is required unless it is in the Stretch section.

Read the whole document once before writing any code.

PROJECT	`CardVault.Api`, an ASP.NET Core Web API using controllers
DATABASE	SQLite file `cardvault.db` managed by EF Core migrations
TABLE	One table, `Cards`, holding `Card` rows. No relationships to other tables.
LAYERS	`CardsController` → `ICardService` → `ICardRepository` → `AppDbContext`
REPOSITORY	Exactly five methods: `GetAll`, `GetById`, `Create`, `Update`, `Delete`
ENDPOINTS	Six. Full CRUD on `/api/cards` plus one domain endpoint, `GET /api/cards/set/{setCode}`

RULES — Four business rules enforced in the service layer, returning 400 or 409

What this practices

- Wiring the full n-tier chain for one resource and registering every piece with dependency injection.
- Keeping each layer honest: HTTP in the controller, decisions in the service, EF Core in the repository.
- Designing DTOs that differ from the entity on purpose: hidden server-set fields and one computed field.
- Choosing the right status code, especially the line between 400 and 409.
- Writing filtering and rule checks with LINQ in the service, since the repository only hands back the whole table or one row.

/02

REQUIRED ARCHITECTURE

One table, one controller, one service, one repository.

If you add a second table in the stretch goals, it gets its own copy of all four, following the same pattern.

01	→ 02	→ 03	→ 04	→ 05
Controller	**Service**	**Repository**	**DbContext**	**SQLite**
Routes, model binding, status codes. Calls the service interface. Knows nothing about EF Core.	Business rules, validation beyond attributes, mapping entity ↔ DTO, LINQ filtering. Calls the repository interface.	The only place EF Core appears. Five methods, returns entities, no DTOs, no HTTP.	`DbSet<Card>` and the SQLite connection. Nothing else.	The `cardvault.db` file created by migrations.

Folder layout

```
CardVault.Api/
├── Controllers/
│   └── CardsController.cs
├── Services/
│   ├── ICardService.cs
│   └── CardService.cs
├── Repositories/
│   ├── ICardRepository.cs
│   └── CardRepository.cs
├── Models/
│   └── Card.cs
├── DTOs/
│   ├── CreateCardDto.cs
│   ├── UpdateCardDto.cs
│   └── CardResponseDto.cs
├── Data/
│   └── AppDbContext.cs
├── Migrations/                 generated
├── Program.cs
```

The repository contract

REPOSITORIES/ICARDREPOSITORY.CS

```
public interface ICardRepository
{
    IEnumerable<Card> GetAll();
    Card? GetById(int id);
    Card Create(Card card);
    Card Update(Card card);
    bool Delete(int id);
}
```

Every repository you write this term has exactly this shape. `GetAll` is `ToList()` over the whole table. No `async` or `await` anywhere in this project; we have not covered it yet. There are no search or filter methods here on purpose: that work belongs to the service, using LINQ over what `GetAll` returns.

```
├── appsettings.json
└── README.md
```

If we put interfaces in an `Interfaces/` folder in class, do that instead. The point is that every piece exists and is named consistently.

Layer rules — REQUIRED

01	The controller depends only on `ICardService`. It never touches the repository or `AppDbContext`.
02	The service depends only on `ICardRepository`. It never returns an entity to the controller, only DTOs.
03	The repository is the only class that uses `AppDbContext` or any `Microsoft.EntityFrameworkCore` type.
04	All three are registered in `Program.cs` with `AddScoped`, interface to implementation.
05	`GetById` returns `null` and `Delete` returns `false` when the id does not exist. The service turns that into a 404; the repository never throws for a missing row.

/03

DATA MODEL

`string?` means nullable. Everything else is required.

Create `Models/Card.cs` with these properties. Use the C# types shown. Simple limits can be enforced with validation attributes on the DTOs; the service handles anything that needs a comparison or a lookup.

PROPERTY	TYPE	RULES
`Id`	`int`	Primary key, generated by the database
`Name`	`string`	Required, max 120
`SetCode`	`string`	Required, 3 to 5 characters, stored upper-case (e.g. `MKM`, `LTR`)
`ManaCost`	`string?`	Optional, max 30, written like `{2}{G}{G}`
`ManaValue`	`int`	0 through 20
`Color`	`string`	One of `White`, `Blue`, `Black`, `Red`, `Green`, `Colorless`, `Multicolor`
`CardType`	`string`	Required, max 60 (e.g. Creature — Elf Druid)
`Rarity`	`string`	One of `Common`, `Uncommon`, `Rare`, `Mythic`
`Quantity`	`int`	0 or more (copies owned)
`Price`	`decimal`	0 or more (value of one copy)

PROPERTY	TYPE	RULES
`IsFoil`	`bool`	Defaults to `false`

There are no foreign keys or navigation properties in this assignment. The table stands alone.

/04

DTOS

Validation attributes go on the DTOs, not the entity.

A failed attribute becomes an automatic 400.

Clients never see or send the `Card` entity. Three DTO classes in `DTOs/` define the API's shape, plus one more DTO for the sixth endpoint.

`CreateCardDto`	All fields except `Id`.
`UpdateCardDto`	Same shape as the create DTO.
`CardResponseDto`	All fields plus a computed `TotalValue` (`decimal`) equal to `Quantity` times `Price`.
`Sixth endpoint`	`SetSummaryDto` returned by the sixth endpoint: `SetCode`, `CardCount` (rows), `TotalCopies` (sum of `Quantity`), `TotalValue` (sum of each card's `TotalValue`), and a `Cards` list of response DTOs.

COMPUTED `TotalValue` does not exist on the entity or in the database. The service calculates it every time it maps an entity to a response DTO. That mapping should live in one private method in the service so every endpoint returns the same shape.

/05

ENDPOINTS

The sixth endpoint is the one that makes this API *this* API. Build CRUD first, then this.

Six endpoints, all on `CardsController` with `[ApiController]` and `[Route("api/cards")]`. Not every endpoint can produce every error; the error column lists the ones this assignment's rules can trigger.

METHOD	ROUTE	BODY	SUCCESS	ERRORS
GET	`/api/cards`	none	200 list of response DTOs	—
GET	`/api/cards/{id}`	none	200 response DTO	404
POST	`/api/cards`	`CreateCardDto`	201 + Location header	400 · 409
PUT	`/api/cards/{id}`	`UpdateCardDto`	204	400 · 404 · 409
DELETE	`/api/cards/{id}`	none	204	404 · 409

METHOD	ROUTE	BODY	SUCCESS	ERRORS
GET	/api/cards/set/{setCode}	none	200 with a SetSummaryDto	404 when no cards exist for that set code

THE SIXTH

The "what do I own from this set?" lookup. In the service, filter `GetAll()` to the set code (case-insensitive), sort by `Name`, and compute the totals with LINQ: `Count()`, `Sum(c => c.Quantity)`, `Sum(c => c.Quantity * c.Price)`.

Status code conventions

CODE	USE IT WHEN
200 OK	Successful GET, and the sixth endpoint when it returns the changed record
201 Created	Successful POST. Return `CreatedAtAction(nameof(GetById), new { id }, dto)` so the `Location` header is set
204 No Content	Successful PUT and DELETE. No body
400 Bad Request	Validation failed: a required field is missing, a number is out of range, a value is not one of the allowed words
404 Not Found	The id (or lookup value) does not exist
409 Conflict	The request was well-formed but a business rule says no: duplicates, a state that cannot change, a limit that is already reached

/06

BUSINESS RULES

400 the request itself is wrong.

409 the request is fine but the current state says no.

These live in `CardService`. The number on the right is the status code the client must receive when the rule is broken. The controller's job is only to translate the service's answer into that code.

01	The combination of `Name` and `SetCode` must be unique, compared case-insensitively. The same card from a different set is a different row.	**409**
02	`SetCode` must be 3 to 5 characters; convert it to upper-case in the service before saving. `ManaValue` must be 0 through 20, `Quantity` and `Price` must be 0 or more. Use validation attributes on the DTOs for the ranges.	**400**
03	`Color` and `Rarity` must be one of their allowed values. Check these in the service and reject anything else.	**400**
04	A card with copies still in the collection cannot be deleted. Set `Quantity` to 0 with PUT first (you traded them away), then delete.	**409**

PATTERN

Because the repository only offers `GetAll` and `GetById`, every uniqueness check, count, and filter is LINQ in the service: `var all = _repo.GetAll();` then `all.Any(...)`, `all.Count(...)`, `all.Where(...)`. The list is already in memory, so `string.Equals(a, b, StringComparison.OrdinalIgnoreCase)` works for case-insensitive comparisons.

/07

STRETCH GOALS

Stretch work only counts if the README says it exists.

Optional. Only start these once all six endpoints and all four rules work, and note in the README which ones you attempted.

Second table: Deck — STRETCH A

Add a `Deck` model (`Name` unique, `Format` one of `Standard`, `Modern`, `Commander`, `Pauper`, `Colors` as text, `IsComplete`) with its own repository, service, controller, and the same five CRUD endpoints. No relationship to `Card`; it is a separate table. Add `PATCH /api/decks/{id}/complete` that sets `IsComplete` to `true` and returns 409 if it already is.

Trade — STRETCH B

`PATCH /api/cards/{id}/trade` with a body of `{ "quantity": 2 }` that subtracts copies, returning 400 for quantities under 1 and 409 if you do not own that many.

Search — STRETCH C

`GET /api/cards/search?color=&rarity=&maxManaValue=` with every parameter optional and case-insensitive.